

F2: $\tan(x)$

SOEN 6011 — Deliverable 2

Konan Brandon

Student ID: 40373534

Summer 2026

Agenda

- 1 Problem 5: Implementation from Scratch
- 2 Problem 6: Version Control
- 3 Problem 7: Updated Requirements
- 4 Demonstration

What “from scratch” meant

- No `math` module. π is a literal constant.
- No factorial, no built-in trigonometry.
- Permitted: conversion, arithmetic, output, interface.

Six subordinate functions had to be written:

```
validate_number  degrees_to_radians  _nearest_integer  
reduce_range    sine / cosine        compute_tan
```

Series without factorials

$$\sin(x) = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \dots$$

Dividing one term by the previous:

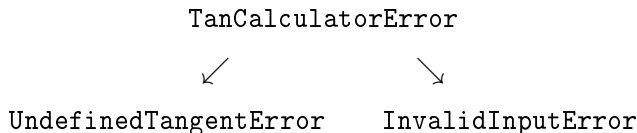
$$\frac{x^5/5!}{x^3/3!} = \frac{x^2}{4 \cdot 5} \implies t_{k+1} = t_k \cdot \frac{-x^2}{(2k)(2k+1)}$$

- One multiply and one divide per term.
- No factorial, no power, nothing overflows.
- Stops when $|t_k| < 10^{-15}$ — not at a fixed count.

Range reduction: why π , not 2π

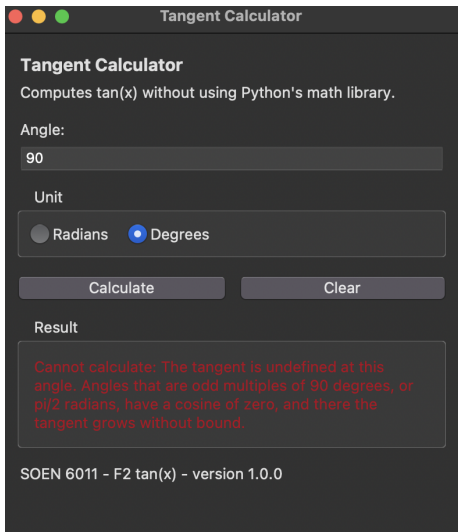
- sin and cos both change sign over π — the signs cancel in the ratio.
- So $\tan(x + \pi) = \tan(x)$: the period is π .
- Reducing by π lands the argument in $[-\pi/2, \pi/2]$, where the series converges fastest.
- It also collapses every asymptote onto one check.

Without this, $x = 1000$ would be unusable.



- One base class, so the interface catches both with a single handler.
- Messages say what to enter instead — never a stack trace.

The graphical interface



- Tkinter; no arithmetic in this module
- Radians or degrees
- Return calculates, Escape clears
- Labels on every control
- Outcome stated in words, not colour alone

Errors the user can act on

Tangent Calculator
Computes $\tan(x)$ without using Python's math library.

Angle:

Unit
☐ Radians ☒ Degrees

Result

That entry is not a number. Please enter a value such as 45 or 0.7854.

SOEN 6011 - F2 $\tan(x)$ - version 1.0.0

Asymptote

Tangent Calculator
Computes $\tan(x)$ without using Python's math library.

Angle:

Unit
☒ Radians ☐ Degrees

Result

That entry is not a number. Please enter a value such as 45 or 0.7854.

SOEN 6011 - F2 $\tan(x)$ - version 1.0.0

Non-numeric

Tangent Calculator
Computes $\tan(x)$ without using Python's math library.

Angle:

Unit
☒ Radians ☐ Degrees

Result

Cannot calculate: That angle is too large to give a trustworthy answer. Please enter a value between -1000000 and 1000000. Beyond that range the angle cannot be reduced precisely enough for the result to be accurate to the number of decimal places shown.

SOEN 6011 - F2 $\tan(x)$ - version 1.0.0

Out of range

Verification: 44 of 44 checks

Angle (rad)	This program	Rel. diff.
$\pi/4$	1.0000000000	1.1×10^{-16}
2.0	-2.1850398633	2.0×10^{-16}
1000.0	1.4703241557	3.8×10^{-14}
1.57	1255.7655915007	3.5×10^{-14}
10^6	-0.3736244539	9.4×10^{-11}

Reference: `math.tan`, imported only in the test harness — never in the implementation.

A defect that reading the code would not reveal

`tan(1015)`

Returned -1.557 . True value -1.672 .

Wrong in the first decimal — and **no error raised**.

Magnitude	Typical digits	Worst
10^4	11.7	8.8
10^6	9.7	6.8
10^8	7.7	5.1

Ten decimal places are displayed. Past 10^6 the worst case falls below six correct digits — so 10^6 is the limit, and larger angles are refused with an explanation.

Two faults found by testing, not by reading

- ① **Silent precision loss** at large arguments. Found by comparing against a reference; fixed by a documented validated range.
- ② **Signature mismatch** between the interface and the calculation module — two arguments passed where one was defined. Each file was correct alone and wrong together.

Exhaustive check: all 636,620 asymptotes in range correctly detected.

`https://github.com/Brand0-code/soen6011-tan-calculator`

- Public; tagged v1.0.0 (Semantic Versioning)
- Eight commits, made as the work progressed
- Imperative mood; the body states *why*
- README: usage, verification, accuracy, validated range
- .gitignore keeps platform artefacts out

Commit messages

```
Add verification harness comparing output against math.tan
```

```
Checks reference angles, degree conversion, range reduction,  
behaviour near asymptotes, undefined points, rejected input,  
and the boundary of the accepted range. The math import is  
confined to this harness and is absent from the implementation.
```

What D1 feedback said, and what changed

D1: REQ-001 ... REQ-008

One flat sequence mixing “compute the tangent” with “six decimal places”.

D2: four categories, each with its own sequence

TAN-FR- <i>nn</i>	functional (12)
TAN-UI- <i>nn</i>	interface (8)
TAN-NFR- <i>nn</i>	quality attributes (6)
TAN-CON- <i>nn</i>	constraints (3)

Change log

Status	Count	Example
Unchanged	4	REQ-001 → TAN-FR-01
Modified	4	REQ-004 CLI → TAN-UI-01 GUI
Added	21	TAN-FR-11 validated range
Removed	0	REQ-004 superseded, not withdrawn

REQ-003 was *split*: plain language is a quality attribute (TAN-NFR-01); “no stack trace” is interface behaviour (TAN-UI-07).

Why 8 requirements became 29

Not padding — a consequence of Problem 5.

- D1 could leave series evaluation, range reduction and termination unstated, because a library did that work.
- Prohibiting the library makes each of those a design decision.
- Every decision carries its own verifiable requirement.

Live demonstration

- ① $45^\circ \rightarrow 1.0000000000$
- ② 1000 radians $\rightarrow$ range reduction works
- ③ $90^\circ \rightarrow$ undefined, explained
- ④ abc $\rightarrow$ not a number
- ⑤ $1e7 \rightarrow$ beyond validated range

`python tan_gui.py` — no IDE, no build step

Summary

- Tangent computed from first principles; no library arithmetic
- Tkinter interface, separated from the calculation
- 44 of 44 verification checks pass; Flake8 clean; Pylint 10.00/10
- Validated range set by measurement, not assumption
- Public repository, tagged v1.0.0
- Requirements restructured to ISO/IEC/IEEE 29148

Thank you — questions?